topic_05_vector_internals

Theory, Edge Cases & Code Examples

- ✓ What is std::vector?
- ✓ Size vs Capacity
- ✓ Growth Strategies
- ✓ Exception Safety
- ✓ Iterator Invalidation

→ Swipe to learn →

topic_05_vector_internals

Theory

📌 1. What is std::vector?

`std::vector` is a dynamic array that automatically manages memory and resizes when needed.

Key characteristics:

- Contiguous memory (cache-friendly)
- Amortized O(1) push_back
- Random access O(1)
- Automatic growth

Internal representation:

```
template<typename T>
class vector {
private:
    T* data_;       // Pointer to dynamic array
    size_t size_;     // Number of elements
    size_t capacity_; // Allocated space
};
```

Visual:

```
capacity_ = 8
size_ = 5

data_ → [1][2][3][4][5][?][?][?]
         ↑            ↑            ↑
      begin()     end()      capacity
```

---

topic_05_vector_internals

Theory

📌 2. Size vs Capacity

Size: Number of constructed elements
Capacity: Total allocated space (may be larger than size)

```
std::vector<int> v;
v.reserve(10);     // capacity = 10, size = 0
v.push_back(42);   // capacity = 10, size = 1
```

Why separate?
- Avoid reallocation on every push_back
- Amortize allocation cost

---

topic_05_vector_internals

Theory

📌 3. Growth Strategies

When `push_back()` exceeds capacity:

1. Allocate larger buffer (typically 1.5× or 2× current capacity)
2. Move/copy elements to new buffer
3. Destroy elements in old buffer
4. Deallocate old buffer

Common growth factors:

Factor	Pros	Cons
2×	Simple, fast growth	More wasted space
1.5×	Less wasted space	Slower growth, may reuse memory
φ (1.618)	Optimal for memory reuse	Complex

libstdc++ uses 2×, libc++ uses 1.5×

Why not 1.1× or exact size?

- Too many reallocations
- Poor amortized complexity

---

topic_05_vector_internals

Theory

📌 4. Exception Safety

Strong exception guarantee:

- If operation fails, vector remains unchanged

Challenges:

```
void push_back(const T& value) {
    if (size_ == capacity_) {
        reallocate();  // May throw (allocation)
    }

    new (data_ + size_) T(value);  // May throw (copy constructor)
    ++size_;
}
```

If copy throws after reallocation:

- Old buffer already destroyed
- Cannot recover

Solution: Use move if `noexcept`, else copy:

```
if constexpr (std::is_nothrow_move_constructible_v<T>) {
    // Move elements (fast, noexcept)
} else {
    // Copy elements (slow, but can rollback)
}
```

---

topic_05_vector_internals

Theory

📌 5. Iterator Invalidation (1/9)

Rules:

- **Reallocation invalidates all iterators** (data_ pointer changes)
- **Insertion/erasure invalidates iterators after modification point**

```
std::vector<int> v = {1, 2, 3};
auto it = v.begin();

v.push_back(4);  // May reallocate
*it;  // ← DANGER: it may be invalidated!
```

How to check if reallocation occurred:

```
auto* old_data = v.data();
v.push_back(x);
if (v.data() != old_data) {
    // Reallocation happened
}
```

```
#include <memory>
#include <algorithm>
#include <stdexcept>
#include <initializer_list>
#include <type_traits>

template<typename T>
class Vector {
private:
    T* data_;
    size_t size_;
    size_t capacity_;

    // ============================================================
    // HELPER: Reallocate with new capacity
    // ============================================================

    void reallocate(size_t new_capacity) {
        // Allocate new buffer
        T* new_data = static_cast<T*>(
// ...continues on next slide
```

📌 5. Iterator Invalidation (2/9)

```
// ...continued from previous slide
            ::operator new(new_capacity * sizeof(T))
        );

        // Move or copy elements
        if constexpr (std::is_nothrow_move_constructible_v<T>) {
            // Move (noexcept - no rollback needed)
            std::uninitialized_move_n(data_, size_, new_data);
        } else {
            // Copy (may throw - use try-catch for rollback)
            size_t i = 0;
            try {
                for (; i < size_; ++i) {
                    new (new_data + i) T(data_[i]);
                }
            } catch (...) {
                // Rollback: destroy constructed elements
                for (size_t j = 0; j < i; ++j) {
                    new_data[j].~T();
                }
                ::operator delete(new_data);
                throw;  // Re-throw
            }
        }

        // Destroy old elements
        for (size_t i = 0; i < size_; ++i) {
            data_[i].~T();
        }

        // Deallocate old buffer
        ::operator delete(data_);

        // Update pointers
        data_ = new_data;
        capacity_ = new_capacity;
    }

public:
    // ============================================================
    // CONSTRUCTORS
// ...continues on next slide
```

📌 5. Iterator Invalidation (3/9)

```
// ...continued from previous slide
    // ==============================================================

    Vector()
        : data_(nullptr), size_(0), capacity_(0) {}

    explicit Vector(size_t count, const T& value = T())
        : data_(nullptr), size_(0), capacity_(0)
    {
        reserve(count);
        for (size_t i = 0; i < count; ++i) {
            push_back(value);
        }
    }

    Vector(std::initializer_list<T> init)
        : data_(nullptr), size_(0), capacity_(0)
    {
        reserve(init.size());
        for (const auto& elem : init) {
            push_back(elem);
        }
    }

    // Copy constructor
    Vector(const Vector& other)
        : data_(nullptr), size_(0), capacity_(0)
    {
        reserve(other.size_);
        for (size_t i = 0; i < other.size_; ++i) {
            push_back(other.data_[i]);
        }
    }

    // Move constructor
    Vector(Vector&& other) noexcept
        : data_(other.data_),
          size_(other.size_),
          capacity_(other.capacity_)
    {
        other.data_ = nullptr;
// ...continues on next slide
```

📌 5. Iterator Invalidation (4/9)

```
// ...continued from previous slide
        other.size_ = 0;
        other.capacity_ = 0;
    }

    // ============================================================
    // DESTRUCTOR
    // ============================================================

    ~Vector() {
        // Destroy all elements
        for (size_t i = 0; i < size_; ++i) {
            data_[i].~T();
        }

        // Deallocate buffer
        ::operator delete(data_);
    }

    // ============================================================
    // ASSIGNMENT OPERATORS
    // ============================================================

    Vector& operator=(const Vector& other) {
        if (this != &other) {
            Vector temp(other);  // Copy
            swap(temp);          // Swap (noexcept)
        }
        return *this;
    }

    Vector& operator=(Vector&& other) noexcept {
        if (this != &other) {
            swap(other);
        }
        return *this;
    }

    // ============================================================
    // CAPACITY
    // ============================================================
// ...continues on next slide
```

📌 5. Iterator Invalidation (5/9)

```
// ...continued from previous slide

    size_t size() const noexcept {
        return size_;
    }

    size_t capacity() const noexcept {
        return capacity_;
    }

    bool empty() const noexcept {
        return size_ == 0;
    }

    void reserve(size_t new_capacity) {
        if (new_capacity <= capacity_) {
            return;  // No-op if already sufficient
        }

        reallocate(new_capacity);
    }

    void shrink_to_fit() {
        if (size_ < capacity_) {
            reallocate(size_);
        }
    }

    // ============================================================
    // ELEMENT ACCESS
    // ============================================================

    T& operator[](size_t index) {
        return data_[index];  // No bounds check (like std::vector)
    }

    const T& operator[](size_t index) const {
        return data_[index];
    }

    T& at(size_t index) {
// ...continues on next slide
```

📌 5. Iterator Invalidation (6/9)

```
// ...continued from previous slide
        if (index >= size_) {
            throw std::out_of_range("Vector::at: index out of range");
        }
        return data_[index];
    }

    const T& at(size_t index) const {
        if (index >= size_) {
            throw std::out_of_range("Vector::at: index out of range");
        }
        return data_[index];
    }

    T& front() {
        return data_[0];
    }

    const T& front() const {
        return data_[0];
    }

    T& back() {
        return data_[size_ - 1];
    }

    const T& back() const {
        return data_[size_ - 1];
    }

    T* data() noexcept {
        return data_;
    }

    const T* data() const noexcept {
        return data_;
    }

    // ============================================================
    // MODIFIERS
    // ============================================================
// ...continues on next slide
```

📌 5. Iterator Invalidation (7/9)

```
// ...continued from previous slide

    void push_back(const T& value) {
        if (size_ == capacity_) {
            size_t new_capacity = (capacity_ == 0) ? 1 : capacity_ * 2;
            reserve(new_capacity);
        }

        new (data_ + size_) T(value);  // Placement new
        ++size_;
    }

    void push_back(T&& value) {
        if (size_ == capacity_) {
            size_t new_capacity = (capacity_ == 0) ? 1 : capacity_ * 2;
            reserve(new_capacity);
        }

        new (data_ + size_) T(std::move(value));
        ++size_;
    }

    template<typename... Args>
    void emplace_back(Args&&... args) {
        if (size_ == capacity_) {
            size_t new_capacity = (capacity_ == 0) ? 1 : capacity_ * 2;
            reserve(new_capacity);
        }

        new (data_ + size_) T(std::forward<Args>(args)...);
        ++size_;
    }

    void pop_back() {
        if (size_ > 0) {
            --size_;
            data_[size_].~T();
        }
    }

    void clear() noexcept {
// ...continues on next slide
```

📌 5. Iterator Invalidation (8/9)

```
// ...continued from previous slide
        for (size_t i = 0; i < size_; ++i) {
            data_[i].~T();
        }
        size_ = 0;
    }

    void resize(size_t new_size, const T& value = T()) {
        if (new_size < size_) {
            // Shrink: destroy excess elements
            for (size_t i = new_size; i < size_; ++i) {
                data_[i].~T();
            }
            size_ = new_size;
        } else if (new_size > size_) {
            // Grow: construct new elements
            reserve(new_size);
            for (size_t i = size_; i < new_size; ++i) {
                new (data_ + i) T(value);
            }
            size_ = new_size;
        }
    }

    void swap(Vector& other) noexcept {
        std::swap(data_, other.data_);
        std::swap(size_, other.size_);
        std::swap(capacity_, other.capacity_);
    }

    // ============================================================
    // ITERATORS (simplified)
    // ============================================================

    using iterator = T*;
    using const_iterator = const T*;

    iterator begin() noexcept {
        return data_;
    }

// ...continues on next slide
```

📌 5. Iterator Invalidation (9/9)

[Code continues across slides]

```
// ...continued from previous slide
    const_iterator begin() const noexcept {
        return data_;
    }

    iterator end() noexcept {
        return data_ + size_;
    }

    const_iterator end() const noexcept {
        return data_ + size_;
    }
};
```

topic_05_vector_internals

Edge Cases

📌 1. Push After Reserve

Behavior: `reserve()` doesn't construct elements, only allocates.

```
Vector<int> v;
v.reserve(10);
std::cout << v.size();     // 0 (not 10!)
std::cout << v[5];         // ← UNDEFINED BEHAVIOR (no element there)

// Correct:
v.resize(10);              // Constructs 10 default elements
std::cout << v[5];         // 0
```

---

topic_05_vector_internals

Edge Cases

📌 2. Move-Only Types

Problem: Types like `std::unique_ptr` cannot be copied.

```
Vector<std::unique_ptr<int>> v;
v.push_back(std::make_unique<int>(42));  // ✓ Move

// Reallocation:
// - Must move elements (not copy)
// - Handled by std::is_nothrow_move_constructible check
```

---

topic_05_vector_internals

Edge Cases

📌 3. Self-Assignment

```
v = v;  // ← Must handle safely
```

Our implementation is safe:

```
Vector& operator=(const Vector& other) {
    if (this != &other) {  // ← Self-assignment check
        Vector temp(other);
        swap(temp);
    }
    return *this;
}
```

topic_05_vector_internals

Edge Cases

📌 4. Reallocating During Iteration

Problem: Reallocation invalidates iterators.

```
Vector<int> v = {1, 2, 3};
for (auto it = v.begin(); it != v.end(); ++it) {
    v.push_back(*it * 2);  // ← Reallocation! it invalidated
}
```

Solution: Don't modify vector during iteration, or cache `end()`:

```
size_t original_size = v.size();
for (size_t i = 0; i < original_size; ++i) {
    v.push_back(v[i] * 2);
}
```

---

topic_05_vector_internals

Edge Cases

📌 5. Exception During Reallocation

Scenario:

1. Allocate new buffer ✓
2. Copy element 5 → **throws exception**
3. Old buffer already destroyed ← **data loss!**

Solution: Strong exception guarantee via try-catch (in `reallocate()`):

```
size_t i = 0;
try {
    for (; i < size_; ++i) {
        new (new_data + i) T(data_[i]);
    }
} catch (...) {
    // Rollback: destroy constructed elements
    for (size_t j = 0; j < i; ++j) {
        new_data[j].~T();
    }
    ::operator delete(new_data);
    throw;  // Old buffer still intact
}
```

---

topic_05_vector_internals

Code Examples

📌 1. Basic Usage (1/2)

This example demonstrates vector's growth strategy through observation of size and capacity changes during sequential push_back operations.

What this code does:
- Starts with an empty vector (size=0, capacity=0)
- Pushes 10 integers sequentially
- After each push, prints both size (number of elements) and capacity (allocated space)
- Shows how capacity grows exponentially (doubling) rather than linearly

Key concepts demonstrated:
- **Size vs capacity distinction**: Size tracks constructed elements, capacity tracks allocated space
- **Amortized O(1) push_back**: Capacity doubles geometrically (1→2→4→8→16), avoiding frequent reallocations
- **Growth factor of 2×**: After initial capacity of 1, each reallocation doubles capacity
- **Spare capacity**: Capacity often exceeds size to accommodate future insertions without reallocation
- **Reallocation triggers**: Capacity increases when size reaches capacity (at pushes 0, 1, 2, 4, 8)

Real-world applications:
- Understanding when your vector code will trigger expensive reallocations
- Deciding whether to call `reserve()` upfront for known sizes
- Debugging performance issues related to repeated reallocations
- Capacity planning for memory-constrained environments

Why this matters:
- **Performance**: Each reallocation costs O(n) to copy elements, but happens only log(n) times
- **Memory efficiency**: Trade-off between wasted space (unused capacity) and reallocation frequency
- **Predictability**: Growth pattern is deterministic, enabling performance predictions

Performance implications:
- Total insertions: 10
- Reallocations: 4 (at pushes 0, 1, 2, 4, 8)
- Total elements copied during growth: 1+2+4+8 = 15 (amortized cost is O(1) per insertion)

Output:

Observation: Capacity doubles (growth factor = 2).

---

topic_05_vector_internals

Code Examples

📌 1. Basic Usage (2/2)

```
#include <iostream>

int main() {
    Vector<int> v;

    std::cout << "Initial size: " << v.size() << '\n';
    std::cout << "Initial capacity: " << v.capacity() << '\n';

    for (int i = 0; i < 10; ++i) {
        v.push_back(i);
        std::cout << "After push " << i << ": size=" << v.size()
                  << ", capacity=" << v.capacity() << '\n';
    }

    return 0;
}
```

```
['Initial size: 0\nInitial capacity: 0\nAfter push 0: size=1, capacity=1\nAfter push 1: size=2, capacity=2\nAfter push
```

topic_05_vector_internals

Code Examples

📌 2. Reserve to Avoid Reallocations (1/2)

This example benchmarks the dramatic performance difference between vectors with and without pre-allocation using reserve().

What this code does:
- **Without reserve**: Creates vector, pushes 1M elements, suffers ~20 reallocations ($\log_2(1{,}000{,}000) \approx 20$)
- **With reserve**: Pre-allocates space for 1M elements, then pushes without any reallocation
- Measures wall-clock time for both approaches using high-resolution clock
- Demonstrates that reserve() provides ~2.5× speedup by eliminating reallocation overhead

Key concepts demonstrated:
- **Reserve vs resize**: `reserve()` only allocates, doesn't construct elements; `resize()` constructs them
- **Amortized cost in practice**: Without reserve, 20 reallocations × O(n) copy = significant overhead
- **Single allocation benefit**: One 1M-element allocation is cheaper than 20 geometric reallocations
- **Copy/move cost**: Each reallocation copies all existing elements to new buffer
- **Memory fragmentation**: Multiple reallocations can fragment heap, reserve() uses single contiguous block

Real-world applications:
- Loading data from files where final size is known (e.g., JSON array with length header)
- Building result vectors in loops where iterations are known upfront
- Reducing jitter in real-time systems (predictable memory allocation)
- Optimizing hot loops in performance-critical code

Why this matters:
- **When to use reserve()**: Known or estimable final size, performance-critical code paths
- **When not to use reserve()**: Size unknown, memory-constrained (over-allocation wastes space)
- **Trade-off**: Upfront memory cost vs incremental allocation

Performance implications:
- Without reserve: 87ms (20 reallocations, ~1.9M elements copied total)
- With reserve: 34ms (0 reallocations, 0 elements copied)
- Speedup: 2.5× (even more dramatic with larger types than int)
- For move-only types: Even bigger win (no copy fallback during reallocation)

Output:

Speedup: 2.5× faster!

---

topic_05_vector_internals

Code Examples

📌 2. Reserve to Avoid Reallocations (2/2)

```
#include <iostream>
#include <chrono>

void benchmark_no_reserve() {
    auto start = std::chrono::high_resolution_clock::now();

    Vector<int> v;
    for (int i = 0; i < 1000000; ++i) {
        v.push_back(i);  // Many reallocations
    }

    auto end = std::chrono::high_resolution_clock::now();
    auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(end - start).count();

    std::cout << "No reserve: " << ms << " ms\n";
}

void benchmark_with_reserve() {
    auto start = std::chrono::high_resolution_clock::now();

    Vector<int> v;
    v.reserve(1000000);  // One allocation
    for (int i = 0; i < 1000000; ++i) {
        v.push_back(i);
    }

    auto end = std::chrono::high_resolution_clock::now();
    auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(end - start).count();

    std::cout << "With reserve: " << ms << " ms\n";
}

int main() {
    benchmark_no_reserve();
    benchmark_with_reserve();

    return 0;
}
```

```
['No reserve: 87 ms\nWith reserve: 34 ms']
```

topic_05_vector_internals

Code Examples

📌 3. Emplace vs Push (1/2)

This example demonstrates the performance difference between push_back (construct then move) and emplace_back (construct in-place).

What this code does:
- Defines `Person` class with instrumented constructors to track construction/copy/move
- **push_back**: Constructs temporary Person("Alice", 30), then moves into vector
- **emplace_back**: Constructs Person directly in vector's allocated memory using constructor args
- Prints construction events showing extra move operation with push_back

Key concepts demonstrated:
- **Perfect forwarding**: emplace_back forwards constructor arguments directly to in-place construction
- **Placement new**: emplace_back uses `new (data_ + size_) T(std::forward<Args>(args)...)`
- **Copy elision limitations**: Even with RVO, push_back still requires move constructor call
- **Move vs copy**: For move-only types (std::unique_ptr), push_back won't compile, emplace_back works
- **Constructor selection**: emplace_back calls exact constructor, push_back creates temporary first

Real-world applications:
- Inserting non-copyable types (unique_ptr, thread objects)
- Avoiding expensive copies/moves for large objects (matrices, large strings)
- Building containers of objects with complex constructors
- Performance-critical code where every allocation counts

Why this matters:
- **Performance**: Eliminates one object construction (especially important for expensive constructors)
- **Enabling move-only**: Only way to insert non-copyable types into vectors
- **Memory efficiency**: One fewer temporary object allocation on stack/heap
- **API clarity**: Constructor arguments at call site instead of temporary object creation

Performance implications:
- push_back: 2 constructions (temporary + move construction in vector)
- emplace_back: 1 construction (direct construction in vector)
- For types with expensive constructors (string, vector): Significant savings
- For trivial types (int, double): Negligible difference

When to use emplace_back:
- Non-copyable types (required)
- Types with expensive construction (recommended)
- When passing constructor arguments directly (clearer code)

When push_back is fine:

topic_05_vector_internals

Code Examples

📌 3. Emplace vs Push (2/2)

```
#include <iostream>
#include <string>

struct Person {
    std::string name;
    int age;

    Person(std::string n, int a) : name(std::move(n)), age(a) {
        std::cout << "Person(" << name << ", " << age << ") constructed\n";
    }

    Person(const Person& other) : name(other.name), age(other.age) {
        std::cout << "Person copied\n";
    }

    Person(Person&& other) noexcept : name(std::move(other.name)), age(other.age) {
        std::cout << "Person moved\n";
    }
};

int main() {
    Vector<Person> v;

    std::cout << "=== Using push_back ===\n";
    v.push_back(Person{"Alice", 30});  // Constructs temp, then moves

    std::cout << "\n=== Using emplace_back ===\n";
    v.emplace_back("Bob", 25);  // Constructs directly in vector

    return 0;
}
```

```
['=== Using push_back ===\nPerson(Alice, 30) constructed  ← Temporary\nPerson moved                  ← Moved into vect
```

What You Learned

Key Takeaways

- ✓ What is std::vector?
- ✓ Size vs Capacity
- ✓ Growth Strategies
- ✓ Exception Safety

→ Swipe to learn →